Self-Adjusting Network Slicing for Dynamic Heterogeneous Task Offloading in UAV-Enabled Mobile Edge Computing

Xulong Li, *Student Member, IEEE*, Wencan Mao, *Member, IEEE*, Xinyi Xu, *Member, IEEE*, Yaxi Liu, *Member, IEEE*, Huifeng Zhang, Wei Huangfu, *Member, IEEE*, and Keping Long, *Senior Member, IEEE*

***Abstract*—Uncrewed aerial vehicle (UAV)-enabled mobile edge computing (MEC) has received wide attention for its ability to significantly reduce latency with flexible mobile aerial nodes. However, the existing works merely consider a fixed periodic network slicing and usually neglect the characterization of computing tasks in practical applications. Motivated by this, we envision a novel scenario where the dynamics and heterogeneity of users and tasks are considered in UAV-enabled MEC networks. We propose a novel self-adjusting two-timescale network slicing scheme based on the internal mechanism in deep reinforcement learning (DRL) to automatically determine whether and when network slicing needs to be reconstructed. Such a novel scheme balances reconstruction cost and system performance compared with traditional fixed schemes. We build an optimization to minimize total system cost for joint UAV trajectory planning and resource allocation at both network and user slice levels. To tackle this problem, we design a novel self-adjusting two-timescale proximal policy optimization (PPO) algorithm that utilizes the property of the critic network, referred to as SaTPPO. Experiment results manifest the effectiveness of the proposed SaTPPO, and it has a lower system cost than the traditional fixed schemes. Moreover, the proposed SaTPPO outperforms the state-of-the-art baselines concerning effectiveness and efficiency.**

***Index Terms*—Mobile edge computing (MEC), uncrewed aerial vehicle (UAV), network slicing, reinforcement learning (RL), two-timescale optimization.**

I. Introduction

With the development of beyond fifth-generation (B5G) and sixth-generation (6G) mobile communication networks, compute-intensive and latency-critical applications, such as augmented reality, virtual reality, and cloud gaming have been developed rapidly, leading to a dramatic increase in computational demand [1], [2], [3]. Edge computing technology is widely used due to its superiority in low latency and proximity to end users, which enables users to fulfill the above requirements by means of task offloading. On this basis, the fulfillment of these application requirements largely depends on the resources obtained, so task offloading strategy, which assigns the computational and communication resources to the end users, is of great research significance [4], [5].

Edge computing technologies face great challenges in efficiently offloading and processing computational tasks with dynamic and heterogeneous nature. Traditional edge computing servers are usually deployed at fixed base stations. Due to the stationary provisioning of resources, such a paradigm cannot provide efficient computing services for various time-varying demands, such as the ones caused by natural disasters, occasional events, or temporary hotspots [6]. At the same time, the heterogeneous nature of applications exacerbates this challenge. Different applications differ significantly in terms of task size, computational complexity, and quality of service (QoS) requirment, resulting in different demands on network resources. Traditional "one-size-fits-all" network architectures are clearly unable to meet the demand for differentiated services for these heterogeneous tasks.

To effectively address these challenges, edge computing systems are beginning to introduce uncrewed aerial vehicles (UAVs) and network slice (NS) technologies. With their flexible mobility, wide coverage, and low deployment cost, UAVs are ideal airborne computing nodes that can provide flexible and scalable computing services for mobile devices, thus breaking the limitations of traditional edge computing systems with terrestrial base stations [7], [8]. Network slicing technology, an emerging network architecture in the 5G era, allows multiple virtual logical networks to be built on a shared physical network infrastructure. Each logical network can provide customized services for specific types of services or industry users.

By introducing network slicing technology, operators can provide highly flexible, on-demand customized network services based on business requirements to meet the differentiated needs of various types of services, industries and

Received 30 December 2024; revised 8 March 2025 and 8 April 2025; accepted 25 April 2025. Date of publication 2 May 2025; date of current version 19 December 2025. This work was supported in part by the National Key Research and Development Program (Grant No. 2023YFB2903901), in part by the National Natural Science Foundation of China (Grant No. 62301028). The associate editor coordinating the review of this article and approving it for publication was W. Xu. *(Corresponding author: Wei Huangfu.)*

Xulong Li, Xinyi Xu, Yaxi Liu, Wei Huangfu, and Keping Long are with the School of Computer and Communication Engineering, University of Science and Technology Beijing, Beijing 100083, China (e-mail: D202210383@xs.ustb.edu.cn; xinyixu@ustb.edu.cn; yaxi.ustb@gmail.com; huangfuwei@ustb.edu.cn; longkeping@ustb.edu.cn).

Wencan Mao is with the Information Systems Architecture Science Research Division, National Institute of Informatics, Tokyo 101-8430, Japan (e-mail: wencan_mao@nii.ac.jp).

Huifeng Zhang is with the Research Center for High Efficiency Computing Infrastructure, Zhejiang Lab, Hangzhou 311500, China (e-mail: zhanghf@zhejianglab.org).

Digital Object Identifier 10.1109/TCCN.2025.3565942

2332-7731 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:08:48 UTC from IEEE Xplore. Restrictions apply.

users [9]. Incorporating network slicing technology into the edge computing system enables customized network services for different types of tasks. This not only meets diverse network service guarantee requirements, but also significantly improves the efficiency and performance of offloading computing tasks. For example, control tasks with strigent latency requirements can be assigned to low-latency network slices; while monitoring data transmission tasks with demanding bandwidth requirements can be assigned to high-bandwidth slices [10]. However, although finer time-grained network slicing can achieve better system performance in dynamic network environments and computational tasks by timely adjusting the resource allocation of the network slices, their deployment cost is related to the change in the resources that the slices have within two consecutive windows, and this frequent adjustment of the resource allocation of the network slices significantly increases the deployment cost. This contradiction constitutes a major challenge for network slicing in UAV-enabled dynamic task offloading.

To better balance the network slicing granularity and deployment costs, some researchers have proposed a two-timescale optimization strategy where both the long and short scales are considered. On the one hand, finer time-grained network resources for users are allocated on a short timescale to ensure that the QoS requirements of users are satisfied. On the other hand, coarser time-grained network resource allocation on a long timescale is also executed to efficiently reduce the deployment cost caused by the frequent reconfiguration of the network slices with respective to resource allocation [11].

However, there still exist several limitations in the existing works. As far as we know, the existing two-timescale works only consider a fixed periodic network slicing time window. Such a fixed scheme is not suitable for dynamic network environments with time-varying computing tasks where too small a time window leads to unnecessary deployment costs in a slowly changing environment and too large a time window results in unreasonable resource allocation in a rapidly changing environment. Furthermore, the existing works usually neglect the user behavioral variations that lead to the heterogeneity of user application types and the dynamic evolution of application types, which are not adapted to practical applications. This characteristic further aggravates the drawbacks of the above-mentioned fixed scheme. Therefore, there still exists an overlap in the flexible network slicing scheme towards the dynamic user behavior.

Motivated by this, we consider a novel scenario where the mobile users dynamically generate various computing tasks in UAV-enabled mobile edge computing networks. In addition, we propose a self-adjusting two-timescale network slicing scheme based on deep reinforcement learning (DRL) algorithms to flexibly optimize both the user-level resource allocation problem at short timescale and the slice-level one at long timescale. Different from the existing fixed scheme, the proposed novel scheme autonomously determines whether and when network slicing needs to be restructured according to dynamic resources and tasks. The contributions of this work are listed as follows:

1) To the best of our knowledge, we first envision a novel scenario in UAV-enabled MEC networks considering the characteristics of the computing tasks for practical applications, i.e., the heterogeneity of generated computational tasks and dynamics of the evolution of computational task types over time. An optimization for network slicing is established, with the goal of minimizing the total system cost, subject to QoS requirements, to jointly allocate user/slice-level resources and plan UAV trajectories in UAV-enabled MEC networks.
2) We propose a novel self-adjusting two-timescale optimization scheme based on the proximal policy optimization (PPO) algorithm, referred to as SaTPPO, which flexibly allocates the resources at both the network slice level in the long timescale and the user level in the short timescale. Such a novel scheme utilizes the property of the critic network that fits the state-value function to predict the reconfiguration gain of the network slices, thus guiding the network slices to be reconfigured at the appropriate time, and thereby realizing the self-adjustment of the size of the slicing time window.
3) Through simulations in a smart campus scenario, the effectiveness of the proposed scheme in dealing with the problem of heterogeneous dynamic task offloading is demonstrated. Specifically, compared to the current state-of-the-art baseline schemes, i.e., the two-timescale reinforcement learning scheme with a fixed network slicing time window, the proposed scheme demonstrates significant advantages with the lowest system cost. Also, the robustness of the proposed scheme to the effects of various parameters.

The rest of the paper is organized as follows. Section II reviews the related works. Section III describes the system model and formulates the optimization problem. The proposed SaTPPO-based two-timescale optimization scheme is detailed in Section IV. Simulation experiments and analysis are carried out in Section V, and finally conclusions are drawn in Section VI.

II. Related Works

This section reviews the related works in terms of network slicing for task offloading and two-timescale optimization for wireless communication networks.

A. Network Slicing for Task Offloading

Network slicing technology provides customized services for heterogeneous computing tasks by constructing multiple logically independent network slices on the same set of shared physical infrastructures, which effectively meets diverse QoS requirements. Therefore, the combination of network slicing and mobile edge computing techniques has become a promising research hot topic.

Network slicing has been widely applied in various mobile edge computing use cases, including telematics, vehicular edge computing networks, and satellite-ground integrated edge computing networks. In order to provide

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:08:48 UTC from IEEE Xplore. Restrictions apply.

differentiated services for heterogeneous computing tasks in edge computing using network slicing techniques, two frameworks have been proposed [12], [13], and a low-cost adaptive network slicing modeling approach has been proposed [14]. Several works [15], [16], [17] introduced an edge computing task offloading framework based on network slicing techniques for the Telematics scenario and proposed optimization schemes based on heuristic algorithms and DRL respectively, aiming to provide differentiated QoS guarantees for task offloading in high-speed vehicles. A dual timescale optimization framework for network slicing was proposed in [18] for the task offloading problem of satellite-ground integrated edge computing networks.

Network slicing has also been studied at different levels in mobile edge computing, with various optimization objectives, such as latency, QoS, and costs. Alkhourly et al. [19] studied the network slicing problem at the level of computational and storage resources at the edge of the network, and proposed a DRL-based approach to maximize the estimated capacity of acceptable requests for various services. Chiang et al. [20] jointly solved the dynamic slicing extension and task offloading problems from the perspective of service provider profit in multi-tenant e-commerce systems, and proposed a resource slicing scheme based on reinforcement learning and alternating optimization algorithms. Jošilo and Dán [21] investigated a network slicing-based edge computing system, and proposed an approximation algorithm with bounded approximation ratios for joint slice selection and edge resource management in order to minimize the computational task completion time. Zhang et al. [22] proposed a NOMA-assisted MEC network slicing scheme, and a two-step resource allocation and task offloading algorithm based on reinforcement learning and switch matching algorithms to minimize the total energy consumption of the network with maximum delay tolerance.

However, the above works are limited to fixed network slicing time windows and fail to fully consider the heterogeneity of user application types and the dynamic evolution of application types over time due to changes in user behavior.

B. Two-Timescale Optimization for Wireless Communication Networks

Two-timescale optimization, as a state-of-the-art strategy, focuses on considering and responding to changes at different timescales. This approach is especially critical in the complex environment of wireless communication networks, where many parameters and conditions change dynamically over time, and these changes are often distributed over different timescales.

In order to improve the performance of the network and control the cost of the system, the dual-timescale optimization scheme has been widely adopted and applied in wireless communication networks. Through this approach, the network can more accurately adapt to the changes in different timescales, thus realizing more efficient and stable operation. Wang et al. [23] proposed a two-timescale optimization mechanism combining matching theory and DRL for edge heterogeneous networks based on intelligent reflective surfaces (IRS) in order to minimize the long-term energy consumption of the system. Similarly, Zhao et al. [24] proposed a two-timescale optimization scheme for RIS-assisted secure synchronous wireless information and power transmission systems to co-optimize the short-term transmit beamforming vectors of the BS and the long-term phase shifts of the IRS in order to maximize the average worst-case secrecy rate.

Numerious methods, including linear programming, successive convex approximation, and Markov decision process, have been applied for two-timescale optimization for wireless communication networks. In [25], a two-timescale optimization scheme based on linear programming and successive convex approximation methods was proposed for the problem of minimizing the long-term average energy transaction expenditures for smart grid heterogeneous cellular networks with renewable energy sources. In [26], a hierarchical Markov decision process framework was developed and a two-timescale stochastic optimization scheme was proposed for the problem of minimizing the task offloading delay of an integrated satellite-terrestrial network. Ding at al. [27], proposed a resource allocation framework to support ultra-reliable low-latency V2V communications, and the proposed framework includes large-scale and small-scale two-timescale resource optimization schemes.

Besides, Lyapunov optimization is another widely used method for such a two-time slot optimization. Reference [28] constructs a novel two-timescale resource management framework for MEC, which is based on the proposal of an online algorithm based on an improved Lyapunov method online algorithm based on an improved Lyapunov method and a fusion of stochastic rounding and Lagrangian dyadic techniques. Reference [29] proposes a two-timescale based user association and cache placement scheme in order to cope with time-varying content popularity and user location in real-world scenarios. Reference [30] [Dynamic Human Digital Twin Deployment at the Edge for Task Execution: A Two-Timescale Accuracy-Aware Online Optimization] for human digital twins in the framework of end-edge-cloud collaboration A new two-timescale accuracy-aware online optimization method based on an improved Lyapunov method is proposed. He et al. [31] investigated the resource allocation problem in industrial IoT networks with hybrid energy supply and proposed a two-timescale resource allocation scheme based on Lyapunov optimization and alternating direction multiplier. Reference [32] designs an online optimization algorithm using an improved Lyapunov technique that integrates stochastic rounding, Lagrangian method and convex optimization. online optimization algorithm that optimizes microservice deployment with layer sharing on different time scales.

Motivated by the above works, we propose a self-adjusting two-timescale optimization scheme based on the DRL algorithm to optimize the user-level and slice-level resource allocation problems on the long-term and short-term timescales, respectively, which can adapt to the dynamic and heterogeneous task offloading UAV-enabled edge computing.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:08:48 UTC from IEEE Xplore. Restrictions apply.

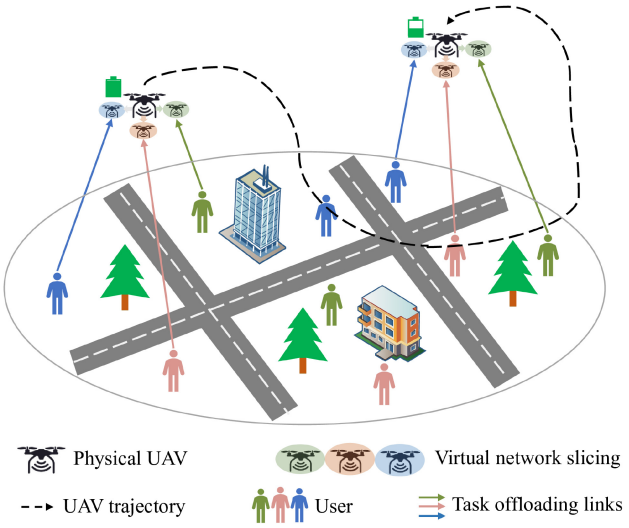


Fig. 1. An exemplary scenario of dynamic task offloading for UAV-enabled edge computing in a smart campus.

III. System Model and Problem Formulation

This section details the exemplary scenario, the system model, and the problem formulation.

A. Exemplary Scenario

In this paper, we focus on the problem of heterogeneous dynamic computational task offloading for UAV-enabled edge computing based on network slicing techniques. Specifically, as shown in Fig. 1, we use a smart campus scenario for demonstration. We consider a square campus area with side length L, where there exists a UAV with an edge server to provide computation offloading services for users (e.g., students and faculty members) carrying smart terminals on the ground. Let $m \in \mathcal{M} = \{1, \ldots, M\}$ represents the set of users.

In the smart campus scenario, there are a large number of heterogeneous smart devices (e.g., mobile terminals of students and employees), and these smart terminals regularly generate heterogeneous computing tasks, which have different data sizes, task complexities, and QoS requirements. In order to provide differentiated services to these heterogeneous computing tasks, we utilize network slicing techniques to slice the UAV-enabled physical network into multiple virtual, independent logical network slices. For example, latency-sensitive tasks with strict latency requirements (e.g., real-time interactive games among students) can be assigned to low-latency network slices; data-intensive tasks with high bandwidth requirements (e.g., transmission of experimental data sets) can be assigned to high-bandwidth slices; and compute-intensive tasks with high computational requirements (e.g., inference for artificial intelligence large models as personal intelligent learning assistants) can be assigned to high-computational resource slices, which helps to realize efficient collaborative work among various intelligent devices in the campus, and provides better technical support for intelligent research, teaching, infotainment, and management.

In addition, we assume that the computing tasks of the smart terminals carried by users are not only heterogeneous but also change dynamically over time. Thus, the entire decision time cycle is discretized into multiple equal time slots, i.e., $t \in \mathcal{T} = \{1, \ldots, T\}$. We assume that smart terminals have limited computational capacity, so they need to offload the generated computational tasks to the UAV's edge servers for processing at each time slot.

B. Network Slicing Model

In the system architecture considered in this paper, the computational tasks generated by smart terminals exhibit significant heterogeneity and dynamism. These tasks vary in terms of computational resource requirements, size of data volume, and sensitivity to latency, and these requirements are constantly changing over time. To effectively cope with this diversity and dynamism, we employ network slicing, a technique that allows multiple virtual logical networks to operate on the same physical network infrastructure.

Specifically, we assume that the computational tasks generated by smart terminals can be categorized into three main types: compute-intensive tasks, which require a large amount of computational resources for processing; data-intensive tasks, which involve the transmission of large-volume data; and latency-sensitive tasks, which have a very high demand on the response time of processing. In order to accurately match these different types of computational tasks, we divide the UAV-enabled physical network resources (including computational resources and bandwidth resources, etc.) into three independent virtual network slices. Each slice is designed for a specific type of task: a network slice $\mathrm{NS_{CI}}$ for compute-intensive tasks, a network slice $\mathrm{NS_{DI}}$ for data-intensive tasks, and a network slice $\mathrm{NS_{DS}}$ for delay-sensitive tasks.

The total available computational resources of the UAV are F. Let $\beta^t_{C,\mathrm{CI}}$, $\beta^t_{C,\mathrm{DI}}$, and $\beta^t_{C,\mathrm{DS}}$ represent the proportion of computational resources allocated to $\mathrm{NS_{CI}}$, $\mathrm{NS_{DI}}$, and $\mathrm{NS_{DS}}$ at time slot t, respectively. Similarly, the total available bandwidth resources of the UAV are B. Let $\beta^t_{B,\mathrm{CI}}$, $\beta^t_{B,\mathrm{DI}}$, and $\beta^t_{B,\mathrm{DS}}$ represent the bandwidth resources allocated to $\mathrm{NS_{CI}}$, $\mathrm{NS_{DI}}$, and $\mathrm{NS_{DS}}$ at time slot t, respectively.

In summary, compared with the UAV-based heterogeneous computing task offloading in existing research [33], [34], network slicing technology provides differentiated and better-quality services by constructing multiple logically independent network slices on the same set of shared physical infrastructure, and each logically isolated network slice is customized by dynamically allocating the corresponding network resources according to the demands of computing tasks. In addition, the network resources of each network slice are logically isolated from each other, effectively slowing down resource competition and seizure between different computing tasks and ensuring that all types of intelligent terminals can obtain the most appropriate computing offload services in the

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:08:48 UTC from IEEE Xplore. Restrictions apply.

corresponding network slices, thus optimizing the performance and efficiency of the entire system.

C. Channel Model

In order to represent the relative position relationship between the UAV, and the smart terminals, we establish a three-dimensional Cartesian coordinate system. Let $\mathbf{Q}_U(t) = \{x_U(t), y_U(t), z_U(t)\}$ denote the coordinates of the position of the UAV at time slot *t*. Let $\mathbf{Q}_I^m(t) = \{x_I^m(t), y_I^m(t), 0\}$ denotes the coordinates of the position of the *m*-th smart terminals. We assume that the UAV can fly within the range of altitudes from z_{min} to z_{max} above the square target area.

The channel between the UAV and the smart terminals is a ground-to-air (A2G) channel. According to [27], we define the channel gain $h_{IU}^m(t)$ between the time slot *t* UAV and the *m*-th smart terminal as

$$h_{IU}^m(t) = \frac{g_T g_R \lambda_C^2}{(4\pi)^2 ||\mathbf{Q}_U(t) - \mathbf{Q}_I^m(t)||_2^p}, \tag{1}$$

where g_T and g_R are the transmit antenna gain of the smart terminal and receive antenna gain of the UAV respectively, λ_C is the wavelength of the signal, and *p* is denoted as the path loss index.

D. Computational Offloading Model

At time slot *t*, the computational task generated by the *m*-th smart terminal is defined as $D_I^m(t) = \{S_I^m(t), C_I^m(t), L_I^m(t), K_I^m(t)\}$, where $S_I^m(t)$ is the data size of the computational task, $C_I^m(t)$ is the number of CPU cycles required to process the computational task, $L_I^m(t)$ is the latency constraint of the computational task, $K_I^m(t)$ is the type of computational task. Let $\mathcal{M}_{\mathrm{CI}}$, $\mathcal{M}_{\mathrm{DI}}$, and $\mathcal{M}_{\mathrm{DS}}$ represent the set of smart terminals whose computational task types are compute-intensive, data-intensive, and delay-sensitive at time slot *t*, respectively, and so $\mathcal{M}_{\mathrm{CI}} \cup \mathcal{M}_{\mathrm{DI}} \cup \mathcal{M}_{\mathrm{DS}} = \mathcal{M}$.

Since the smart terminals do not have sufficient computing power, they need to offload their tasks to the server of the UAV for processing. Therefore, the delay caused by the *m*-th smart terminal offloading the computation task to the UAV's edge server at time slot *t* is

$$T_{\mathrm{tra}}^m(t) = \frac{S_I^m(t)}{R_{IU}^m(t)}, \tag{2}$$

where $R_{IU}^m(t)$ is the data transfer rate of the *m*-th smart terminal.

We assume that all smart terminals use orthogonal frequency division multiple access (OFDMA) technology when transmitting computation tasks to the edge server of the UAV, i.e., they transmit data over different orthogonal subcarriers, so there is no inter-user interference among smart terminals. Therefore, the data transfer rate $R_{IU}^m(t)$ can be written as

$$R_{IU}^m(t) = \beta_{B,*}(t)\alpha_B^m(t) B \log_2\left(1 + \frac{P_I^m h_{IU}^m(t)}{\sigma^2}\right), \quad \forall m \in \mathcal{M}_*, * \in \{\mathrm{CI}, \mathrm{DI}, \mathrm{DS}\} \tag{3}$$

where $\alpha_B^m(t)$ is the proportion of the bandwidth allocated by the UAV to the *m*-th smart terminal to the total available bandwidth of its corresponding network slice, P_I^m is the transmission power of the *m*-th smart terminal, and σ is the Gaussian noise power.

After the computational tasks are offloaded to the UAV, the edge server of the UAV allocates corresponding computational resources to them based on the information of the tasks in order to process them within its latency constraint. At time slot *t*, the latency caused by the edge server of the UAV to process the computational task offloaded by the *m*-th smart terminal is

$$T_{\mathrm{com}}^m(t) = \frac{S_I^m(t) C_I^m(t)}{\beta_{C,*}(t)\alpha_C^m(t) F}, \forall m \in \mathcal{M}_*, * \in \{\mathrm{CI}, \mathrm{DI}, \mathrm{DS}\} \tag{4}$$

where $\alpha_C^m(t)$ is the proportion of computing resources allocated by the UAV to the *m*-th smart terminal to the total available computing resources of its corresponding network slice.

Since the data size of the computation task after it has been processed by the UAV is usually much smaller than the original data, this paper, like most of the existing ones, ignores the latency caused by the return of the computation task from the UAV to the smart terminal. Therefore, at time slot t, the total latency for processing the computation task generated by the *m*-th smart terminals includes both the latency for the smart terminals to offload the computation task to the UAV and the latency for the edge server of the UAV to process the computation task, i.e., $T^m(t) = T_{\mathrm{tra}}^m(t) + T_{\mathrm{com}}^m(t)$.

E. UAV Energy Model

The energy consumption of UAVs mainly consists of three parts: flight energy consumption and computational energy consumption. For UAV flight energy consumption E_U^{fli}, we use the same UAV flight energy model as in [35].

For the computational energy consumption E_U^{comp} of the UAV, which is caused by processing the computational tasks offloaded by the smart terminals, it can be expressed as

$$E_U^{\mathrm{com}}(t) = \sum_{*\in\{\mathrm{CI,DI,DS}\}} \sum_{m\in\mathcal{M}_*} g_U \left(\beta_{C,*}^t \alpha_C^{m,t} F\right)^2 T_{\mathrm{com}}^{m,t}, \tag{5}$$

where g_U is the constant associated with the chip architecture of the UAV.

Therefore, the total energy consumption of the UAV at time slot *t* can be expressed as $E_U(t) = E_U^{\mathrm{fli}}(t) + E_U^{\mathrm{com}}(t)$.

F. System Cost Model

Given that standardized QoS metrics (e.g., sum rate or latency) cannot effectively reflect the critical element of deployment cost of network slicing, it is also difficult to accurately capture the variability among heterogeneous tasks. We will customize the total system cost of UAV network slicing systems in this section, which includes the resource consumption cost, the network slicing reconfiguration cost, the computational task processing failure cost, and the UAV's

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:08:48 UTC from IEEE Xplore. Restrictions apply.

energy consumption cost. We will describe each of them below.

- *Resource consumption cost:* Resource consumption cost is the cost caused by the consumption of bandwidth and computational resources by the UAV when processing computational tasks, which can be expressed as

$$C_{\text{res}}(t) = \sum_{*\in\{\text{CI,DI,DS}\}} \omega_{\text{comm}}\beta_{B,*}(t)B + \omega_{\text{comp}}\beta_{C,*}(t)F, \quad (6)$$

  where ω_{comm} and ω_{comp} are weighting factors.
- *Network slicing reconfiguration cost:* The network slice reconfiguration cost is related to the change in resources owned by the slice over two consecutive windows. Since resource release can be fast and inexpensive, we only consider the cost of network slice reconfiguration and the cost inherent in network slice reconfiguration under the impact of resource increase [36]. Therefore, the network slice reconfiguration cost can be expressed as

$$C_{\text{sli}}(t) = \sum_{*\in\{\text{CI,DI,DS}\}} \Big[\omega_{\text{B}}^{\text{sli}} B\big[\beta_{B,*}(t) - \beta_{B,*}(t-1)\big]^{+} + \omega_{C}^{\text{sli}} F\big[\beta_{C,*}(t) - \beta_{C,*}(t-1)\big]^{+}\Big] + P_{\text{sli}}, \quad (7)$$

  where P_{sli} is the inherent cost of network slice reconfiguration, $\omega_{\text{B}}^{\text{sli}}$ and ω_{C}^{sli} are weighting factors.
- *Computational task processing failure cost:* The UAV needs to make sure that the computational tasks generated by the smart terminals are processed within their latency constraints. The computational task processing failure cost is the penalty when the computational task processing fails, which can be expressed as

$$C_{\text{comp}}(t) = \sum_{m=1}^{M} P_{\text{comp}}\mathbb{I}(T^m(t) > L_I^m(t)), \quad (8)$$

  where P_{comp} is the cost (penalty) of failing to process the computational task.
- *UAV's energy consumption cost:* The energy of a UAV is usually limited, so in order to save energy, we define the UAV's energy consumption cost according to the previous section as $C_{\text{ener}}(t) = E_U(t)$.

Based on the above discussion, the total system cost in time slot t can be given by

$$C_{\text{tot}}(t) = \omega_{\text{res}} C_{\text{res}}(t) + \omega_{\text{sli}} C_{\text{sli}}(t) + \omega_{\text{comp}} C_{\text{comp}}(t) + \omega_{\text{ener}} C_{\text{ener}}(t). \quad (9)$$

where $\omega_{\text{res}}, \omega_{\text{sli}}, \omega_{\text{comp}}$, and ω_{ener} are weighting factors.

G. Problem Formulation

The objective of this paper is to minimize the total cost of the system by optimizing the network slicing cycle management and resource allocation. Therefore, the optimization problem can be formulated as

$$\text{P1:} \ \min_{\mathbb{X}} \sum_{t=0}^{T} C_{\text{tot}}^{t}\left(X^t\right) \quad (10)$$

$$\text{s.t.} \ \ 0 \le x_U(t), y_U(t) \le L, \forall t \in \mathcal{T}, \quad (10a)$$

$$z_{min} \le z_U(t) \le z_{max}, \forall t \in \mathcal{T}, \quad (10b)$$

$$||\mathbf{Q}_U(t) - \mathbf{Q}_U(t-1)|| \le V_U^{max}, \forall t \in \mathcal{T}, \quad (10c)$$

$$\beta_{C,\text{CI}}(t) + \beta_{C,\text{DI}}(t) + \beta_{C,\text{DS}}(t) \le 1, \forall t \in \mathcal{T}, \quad (10d)$$

$$\beta_{B,\text{CI}}(t) + \beta_{B,\text{DI}}(t) + \beta_{B,\text{DS}}(t) \le 1, \forall t \in \mathcal{T}, \quad (10e)$$

$$\sum_{m\in\mathcal{M}_*} \alpha_C^m(t) = 1, \forall t \in \mathcal{T}, * \in \{\text{CI, DI, DS}\}, \quad (10f)$$

$$\sum_{m\in\mathcal{M}_*} \alpha_B^m(t) = 1, \forall t \in \mathcal{T}, * \in \{\text{CI, DI, DS}\}, \quad (10g)$$

$$\sum_{t=1}^{T} E_U(t) \le E_U^{\max}, \quad (10h)$$

where $\mathbb{X} = \{X(t), \forall t \in \mathcal{T}\}$, $X(t) = \{\{\beta_{C,*}(t), \beta_{B,*}(t), \forall * \in \{\text{CI, DI, DS}\}\}, \{\alpha_C^m(t), \alpha_B^m(t), \forall m \in \mathcal{M}\}, \mathbf{Q}_U(t)\}$, V_U^{max} is the maximum displacement of the UAV, and $E_U^{\max}$ is the maximum energy storage for the UAV.

The wireless channel and task requests are dynamically changing, making problem P1 difficult to be solved by traditional optimization algorithms. In addition, P1 is a long-term optimization problem, which makes it more challenging to solve this problem. Therefore, this paper follows by proposing a solution based on reinforcement learning.

IV. Proposed SaTPPO-Based Scheme

This section presents the proposed SaTPPO-based scheme, namely a self-adjusting two-timescale optimization scheme based on the PPO algorithm.

A. Two Timescale Optimization

Since the computing tasks generated by the smart terminals in the scenario considered in this paper are heterogeneous and dynamic, in order to match the resources required by the computing tasks and improve the execution success rate of the computing tasks, the UAV needs to adjust the slicing resources frequently, which will lead to additional slicing overhead. Therefore, in order to reduce the overall cost of the system, we first reconstruct the optimization problem P1 into two optimization sub-problems with different timescales, i.e., at the slice-level and the user-level. This two-timescale optimization approach not only significantly reduces the reconfiguration cost of network slicing, but also helps to simplify the complexity of the problem so that it can be optimized more efficiently for their respective characteristics.

Specifically, the variations in the types of computational tasks generated by smart terminals occur on longer timescales, so the slice-level sub-optimization problem is considered a long-timescale optimization problem. At this level, decisions are made over relatively long time frames with the aim of rationally allocating the corresponding computational and bandwidth resources to individual network slices. Considering the high cost of network slice reconfiguration, this long-term optimization helps to reduce unnecessary resource adjustments and reconfigurations, thus reducing the overall cost.

In contrast, the wireless network environment changes on shorter timescales, so the user-level sub-optimization problem is considered a short-timescale optimization problem. At this level, decisions are made in a relatively short time frame, which means that in a short time interval, the system needs

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:08:48 UTC from IEEE Xplore. Restrictions apply.

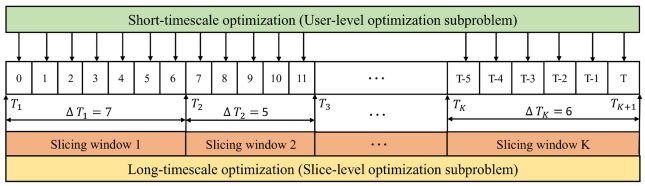


Fig. 2. The two-timescale optimization framework.

to precisely allocate computational and bandwidth resources to each user and simultaneously plan the UAV flight path. This fine-grained management can maximize the efficiency of resource utilization while maintaining service quality. System performance can be optimized by real-time monitoring and dynamic adjustment of resource allocation.

As shown in Fig. 2, there are a total of K network slicing time windows throughout the decision period, where the k-th time window starts from the T_k-th time slot and lasts until the T_{k+1}-th time slot, with a size of $\Delta T_k = T_{k+1} - T_k$. Then, we reconstruct problem P1 into two optimization sub-problems with different timescales in the framework of two-timescale optimization, i.e., the slice-level long-timescale optimization sub-problem

$$\text{P2:} \min_{\mathbb{X}_S} \sum_{k=1}^{K} \sum_{t=T_k}^{T_{k+1}} C_{\text{tot}}(t)(X_S(t)|X_U(t)) \tag{11}$$

$$\text{s.t.} \quad (10e) - (10g), \tag{11a}$$

$$X_S(t) = X_S(T_k), \forall t \in [T_k, T_{k+1}), \tag{11b}$$

and the user-level short-timescale optimization sub-problem

$$\text{P3:} \min_{\mathbb{X}_U} \sum_{k=1}^{K} \sum_{t=T_k}^{T_{k+1}} C_{\text{tot}}(t)(X_U(t)|X_S(t)) \tag{12}$$

$$\text{s.t.} \quad (10a) - (10d), (10g) \tag{12a}$$

$$X_S(t) = X_S(T_k), \forall t \in [T_k, T_{k+1}), \tag{12b}$$

where $\mathbb{X}_S = \{X_S(t), \forall t \in \mathcal{T}\}$, $X_S(t) = \{\beta_{C,*}(t), \beta_{B,*}(t), \forall * \in \{\text{CI}, \text{DI}, \text{DS}\}\}$ is the optimization variable for the slice-level long-timescale sub-problem, $\mathbb{X}_U = \{X_U(t), \forall t \in \mathcal{T}\}$, $X_U(t) = \{\{\alpha_C^m(t), \alpha_B^m(t), \forall m \in \mathcal{M}\}, \mathbf{Q}_U(t), \forall t \in \mathcal{T}\}$ is the optimization variable for user-level short-timescale sub-problem. Constraints (11b) and (12b) indicate that the resource allocation of the network slices is reconfigured at the beginning of each time window and remains constant throughout the time window to minimize the reconfiguration cost of the network slices.

B. Problem Reconstruction in Two Timescales MDP Architectures

In order to solve the above optimization problem with different timescales, this paper first models it as a two-timescale Markov decision process (MDPs) and then proposes a scheme based on two-timescale reinforcement learning.

Specifically, we reconstruct problem P2 and P3 as a two-timescale MDPs which can be represented as the tuple $\{\mathbb{S}, \mathbb{A}_S, \mathbb{A}_U, \mathbb{R}, P\}$, where $\mathbb{S}$ is the state space, $\mathbb{A}_S$ is the long timescale action space for the slice-level optimization sub-problem P2, and $\mathbb{A}_U$ is the short timescale action space for the user-level optimization sub-problem P3, $\mathbb{R}$ is the reward function, and P is the transition dynamics. We will describe each of them below.

- *State space* $\mathbb{S}$*:* The state of the system includes the position of the UAV, the allocation ratio of computational and bandwidth resources for each network slice, the position of the smart terminals, and the information of the computational task. Therefore, the state of the system $s(t) \in \mathbb{S}$ at time slot t can be expressed as

$$s(t) = \{t, \mathbf{Q}_U(t), \{\beta_{C,*}(t), \beta_{B,*}(t), \forall * \in \{\text{CI}, \text{DI}, \text{DS}\}\}, \{\mathbf{Q}_I^m(t), S_I^m(t), C_I^m(t), L_I^m(t), K_I^m(t), \forall m \in \mathcal{M}\}, \tag{13}$$

- *Long timescale action space* $\mathbb{A}_S$*:* The long timescale action of the system is a decision for the slice-level optimization sub-problem P2, which consists of variables for allocating computational and bandwidth resources to individual network slices. Therefore, the long timescale action $a_S(t) \in \mathbb{A}_S$ at time slot t can be expressed as

$$a_S(t) = \{\Delta\beta_{C,*}(t), \Delta\beta_{B,*}(t), \forall * \in \{\text{CI}, \text{DI}, \text{DS}\}\}, \tag{14}$$

- *Short timescale action space* $\mathbb{A}_U$*:* The short timescale actions of the system are decisions for the user-level optimization sub-problem P3, which includes the allocation variables of computational and bandwidth resources for each user, and UAV trajectory planning. Therefore, the short timescale action $a_U(t) \in \mathbb{A}_U$ at time slot t can be expressed as

$$a_U(t) = \{\{\alpha_C^m(t), \alpha_B^m(t), \forall m \in \mathcal{M}\}, \mathbf{Q}_U^t\}. \tag{15}$$

- *Reward function* $\mathbb{R}$*:* According to the optimization problem P1, the optimization objective in this paper is to minimize the total cost of the system, whereas the objective in the MDP is to maximize the reward. Therefore, we define in this paper the system's reward $r(t) \in \mathbb{R}$ at time slot t as the negative of the system's cost, i.e.,

$$r(t) = r(s(t), a(t)) = -C_{\text{tot}}(t). \tag{16}$$

The reason why the multiple constraints (10a)-(10h) in the optimization problem formulation in Eq. (10) are not adequately considered in the design of the reward function of RL is that they can be separately satisfied by. For constraints (10a) and (10b), we satisfy them by truncating the position coordinates of the UAV after it flies out of the boundary. For constraints (10c), we satisfy it by mapping the corresponding portion of the action that is output from the policy network to within the maximum velocity. For constraints (10e) and (10d), We satisfy them by introducing an auxiliary action and then performing the Softmax normalization operation. For constraints (10f) and (10g), we satisfy them by applying the Softmax normalization operation to the outputs of the policy network. For constraints (10h), we satisfy them by applying the Softmax normalization operation to the outputs of the policy network. For constraint 18 (k), we

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:08:48 UTC from IEEE Xplore. Restrictions apply.

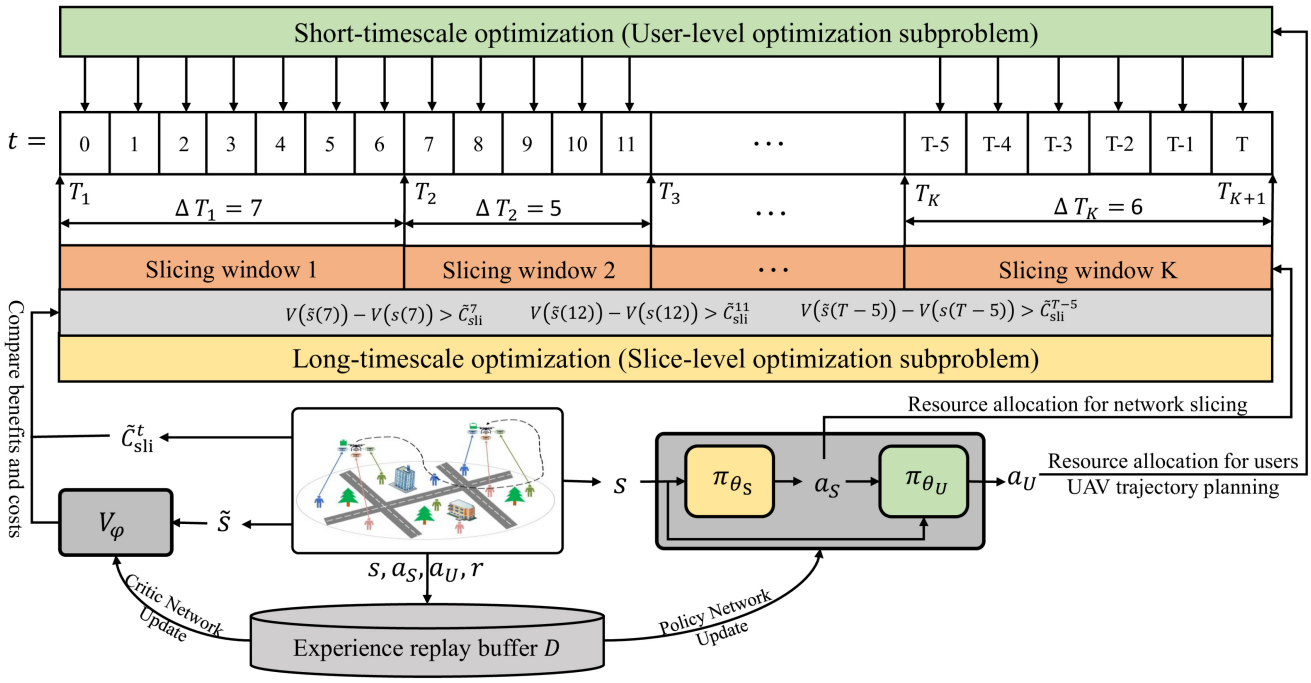


Fig. 3. Framework diagram of the proposed SaTPPO-based scheme.

satisfy it by assuming that the UAVs cease operation once their energy is depleted.

- *Transition dynamics P:* The transition dynamics $p(t) \in P$ is defined as the probability that the agent in state $s(t)$ takes joint action $a(t) = \{a_S(t), a_U(t)\}$ after observation of state $s(t+1)$, i.e.,

$$p(t) = p(s(t+1)|s(t), a(t)). \tag{17}$$

It is worth noting that the transition dynamics are unknown yet observable due to the dynamic nature of the network environment and the computational task.

C. Two Timescales PPO-Based Scheme

In order to solve the above two timescales MDPs, this paper proposes a two timescales reinforcement learning scheme based on PPO.

In the proposed scheme, there is a policy network π_θ with parameter θ and a critic network V_φ with parameter φ, where the policy network π_θ takes state s as input and outputs joint action $a = \{a_S, a_U\}$, and the critic network V_φ is used to fit the expected cumulative discount rewards for an agent's subsequent actions to adopt the policy π in a particular state s, i.e., the state-value function $V_\varphi^\pi(s) = \mathbb{E}[\sum_{t'=t}^{T} \gamma^{t'-t} r(t') | S(t) = s, A(t{:}T) \sim \pi]$, where $\gamma \in [0,1)$ is the discount factor. The policy network π_θ consists of two parts: a long timescale policy network π_{θ^S} with parameter θ^S and a short timescale policy network π_{θ^U} with parameter θ^U, where the long timescale policy network π_{θ^S} takes the state s as an input and outputs the long timescale action a_S; the short timescale policy network π_{θ^U} takes the state s and the long timescale action a_S as an input and outputs the short timescale action a_U.

In addition, there is an experience replay buffer D for storing experience tuples $\langle s, a_S, a_U, r \rangle$ generated by interactions between the agent and the environment. The experience tuples in the experience replay buffer are used to update the agent's policy network and critic network.

The two-timescale PPO is an on-policy algorithm that is less data-efficient, so it utilizes an important sampling technique to reuse the empirical tuples to update the network parameters in order to improve the data efficiency and speed up the training. The policy network optimizes the parameters by maximizing the following truncated objective function

$$J(\theta) = \mathbb{E}\Big[\min\Big\{ d_\theta \hat{A}, \mathrm{clip}(d_\theta, 1-\sigma, 1+\sigma)\hat{A} \Big\}\Big], \tag{18}$$

where $\mathrm{clip}(X, 1-\sigma, 1+\sigma)$ is the clipping function which clips X to the intervals $1-\sigma$ to $1+\sigma$, $d_\theta = \frac{\pi_\theta(a(t)|s(t))}{\pi_{\theta^{old}}(a(t)|s(t))}$ is the probability ratio of old and new policies which is used as the importance sampling ratio, θ^{old} is the parameter of the policy network when the agent interacts with the environment to generate experience tuples in D, $\hat{A}$ is the estimated advantage function, which can be computed using a truncated version of the generalized advantage estimate, i.e.,

$$\begin{aligned} \hat{A}(t) &= \sum_{t'=t}^{T} (\gamma\lambda)^{t'-t} \delta(t') \\ &= \sum_{t'=t}^{T} (\gamma\lambda)^{t'-t} \big[r(t') + \gamma V_\varphi(s(t'+1)) - V_\varphi(s(t')) \big] \end{aligned} \tag{19}$$

where $\lambda \in [0,1]$ is a constant that regulates the trade-off between bias and variance.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:08:48 UTC from IEEE Xplore. Restrictions apply.

According to the Bellman equation, the critic network can update its parameters by minimizing the following mean square error (MSE) loss function

$$L(\varphi) = \mathbb{E}_{\pi}\left[\left(\hat{R}(s(t)) - V_{\varphi}^{\pi}(s(t))\right)^2\right] \tag{20}$$

where $\hat{R}(s(t)) = r(t) + \gamma V_{\varphi}^{\pi}(s(t+1)) = \sum_{t'=t}^{T} \gamma^{t'-t} r(t')$ denotes the cumulative discounted rewards starting from state $s(t)$.

D. Self-Adjusting Network Slicing Scheme

In existing studies, it is usually assumed that the computational tasks generated by smart terminals are heterogeneous. Thus, these studies optimize the slice-level optimization sub-problems for long timescales and the user-level optimization sub-problems for short timescales, respectively, based on two-timescale reinforcement learning. However, existing studies neglect the fact that the types of computational tasks change dynamically over time. Consequently, time window sizes of the network slices in these studies are usually fixed and cannot be dynamically adjusted according to the system state, which has a serious detrimental effect on the system performance.

In the scenario considered in this paper, the computational tasks generated by smart terminals are not only heterogeneous butalso dynamic (i.e., the types of their computational tasks also change dynamically over time). Therefore, UAVs need to reconfigure the parameters of each network slice in a timely and dynamic manner according to the time-varying computational tasks and their computational and bandwidth resources demand. In this way, we can achieve an effective balance between resource utilization and task processing, and improve the success rate of processing computational tasks while avoiding waste of resources. However, reconfiguration of network slices, especially in a frequent manner, significantly increases the operational cost. Hence, the fixed network slicing configuration proposed in the existing schemes has an obvious deficiency in techno-economic performance. To this end, this paper proposes a novel intelligent network slice lifecycle management scheme that dynamically adjusts the time window size of network slices and reconfigures the network slices at the appropriate time.

Specifically, as shown in the flowchart in Fig. 4, the proposed scheme utilizes the property of the critic network V_{φ}^{π} that fits the state-value function to predict the reconfiguration gain of the network slices, thus guiding the network slices to be reconfigured at the appropriate time, and realizing the self-adjustment of the size of the slicing time window. The specific steps are as follows:

1) Use the long timescale policy network π_{θ^S} to output a new long timescale action $\tilde{a}_S$

$$\tilde{a}_S(t) = \{\Delta\tilde{\beta}_{C,*}(t), \Delta\tilde{\beta}_{B,*}(t), \forall * \in \{\mathrm{CI}, \mathrm{DI}, \mathrm{DS}\}\}, \tag{21}$$

and compute a new resource allocation configuration for the network slices

$$\tilde{\beta}_{C,*}(t) = \Delta\tilde{\beta}_{C,*}(t) + \beta_{C,*}(t),$$

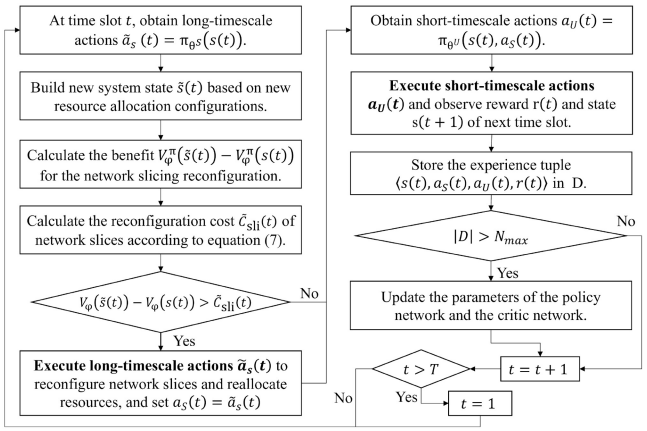


Fig. 4. Flowchart of SaTPPO.

$$\tilde{\beta}_{B,*}(t) = \Delta\tilde{\beta}_{B,*}(t) + \beta_{B,*}(t), \forall * \in \{\mathrm{CI}, \mathrm{DI}, \mathrm{DS}\}. \tag{22}$$

2) Build new system state $\tilde{s}$ based on new resource allocation configurations for network slices.

$$\tilde{s}(t) = \{\mathbf{Q}_U(t), \{\tilde{\beta}_{C,*}(t), \tilde{\beta}_{B,*}(t), \forall * \in \{\mathrm{CI}, \mathrm{DI}, \mathrm{DS}\}\},$$
$$\{\mathbf{Q}_I^m(t), S_I^m(t), C_I^m(t), L_I^m(t), K_I^m(t), \forall m \in \mathcal{M}\}\}, \tag{23}$$

3) Calculate the state-value $V_{\varphi}^{\pi}(s(t))$ for not reconfiguring network slices based on the current state $s(t)$,

$$V_{\varphi}^{\pi}(s(t)) = \mathbb{E}\left[\sum_{t'=t}^{T} \gamma^{t'-t} r(t') | S(t) = s, A(t{:}T) \sim \pi\right], \tag{24}$$

and the state-value $V_{\varphi}^{\pi}(\tilde{s}(t))$ for reconfiguring network slices based on the new state $\tilde{s}(t)$,

$$V_{\varphi}^{\pi}(\tilde{s}(t)) = \mathbb{E}\left[\sum_{t'=t}^{T} \gamma^{t'-t} r(t') | S(t) = \tilde{s}, A(t{:}T) \sim \pi\right], \tag{25}$$

then take the difference (i.e., $V_{\varphi}(\tilde{s}(t)) - V_{\varphi}(s(t))$) between the two as the benefit for the network slicing reconfiguration.

4) Calculate the reconfiguration cost $\tilde{C}_{\mathrm{sli}}^{t}$ of network slices according to equation (7), i.e.,

$$\tilde{C}_{\mathrm{sli}}(t) = \sum_{*\in\{\mathrm{CI},\mathrm{DI},\mathrm{DS}\}} \Big[\omega_{\mathrm{B}}^{\mathrm{sli}} B\left[\tilde{\beta}_{B,*}(t) - \tilde{\beta}_{B,*}(t-1)\right]^{+}$$
$$+\omega_{C}^{\mathrm{sli}} F\left[\tilde{\beta}_{C,*}(t) - \tilde{\beta}_{C,*}(t-1)\right]^{+}\Big] + P_{\mathrm{sli}}, \tag{26}$$

5) Compare the benefits and costs of network slice reconfiguration. Reconfigure when the benefits outweigh the costs (i.e., $V_{\varphi}(\tilde{s}(t)) - V_{\varphi}(s(t)) > \tilde{C}_{\mathrm{sli}}^{t}$) and reallocate resources for network slices, otherwise do not reconfigure and keep the original network slice resource allocation.

Through the above approach, the reconfiguration cost of network slices can be evaluated using the critic network and provide guidance for dynamic and intelligent reconfiguration of network slices, thus realizing the adaptive adjustment of the network slicing time window and the optimal allocation of resources, which reduces the system cost while efficiently

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:08:48 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 Self-Adjusting Two-Timescale PPO

Initialize: Initialize the parameters θ and φ of the policy network π_θ and the critic network V_φ.

for n_{epi} in $\{1, \ldots, N_{epi}\}$ **do**
  Reset initial state $s(1)$.
  $a_S(1) = \pi_{\theta^S}(s(1))$.
  for t in $\{1, \ldots, T\}$ **do**
    $\tilde{a}_S(t) = \pi_{\theta^S}(s(t))$.
    Build new system state $\tilde{s}(t)$ based on new resource allocation configurations for network slices.
    Calculate the benefit $V_\varphi^\pi(\tilde{s}(t)) - V_\varphi^\pi(s(t))$ for the network slicing reconfiguration.
    Calculate the reconfiguration cost $\tilde{C}_{\text{sli}}(t)$ of network slices according to equation (7).
    if $V_\varphi(\tilde{s}(t)) - V_\varphi(s(t)) > \tilde{C}_{\text{sli}}(t)$ **then**
      Execute long-timescale actions $\tilde{a}_S(t)$ to reconfigure network slices and reallocate resources to network slices.
      $a_S(t) = \tilde{a}_S(t)$.
    end if
    $a_U(t) = \pi_{\theta^U}(s(t), a_S(t))$
    Execute actions $a_U(t)$ and observe reward $r(t)$ and state $s(t+1)$ of next time slot.
    Store the experience tuple $\langle s(t), a_S(t), a_U(t), r(t)\rangle$ in D.
    if $|D| > N_{max}$ **then**
      $\theta^{old} \leftarrow \theta$
      Calculate the estimated advantage function $\hat{A}$ based on equation (19).
      for c in $\{1, \ldots, C\}$ **do**
        Sample a random minibatch of tuples from D.
        Update the parameter of policy network by maximizing $J(\theta)$.
        Update the parameter of critic network by minimizing $L(\varphi)$.
      end for
    end if
  end for
end for

handling the computation task. The details of the proposed SaTPPO-based scheme are shown in **Algorithm 1**.

E. Convergence Analysis

The proposed SaTPPO-based scheme belongs to a policy iteration-based reinforcement learning algorithm, whose learning process primarily involves alternating between policy evaluation and policy improvement phases.

During the policy evaluation phase, the value function V_φ^π is computed for policy π. The proposed scheme learns a value function through the Bellman equation (equation (20)), which enables the solution of the two-timescale MDPs via dynamic programming methods. Based on the Bellman equation, we define the Bellman operator $\mathcal{B}_\pi V_\varphi^\pi(s)$ as follows

$$\mathcal{B}_\pi V_\varphi^\pi(s) \coloneqq \mathbb{E}_{a\sim\pi}[r(s,a) + \gamma \mathbb{E}_{s'\sim p(s'|s.a)}\big[V_\varphi^\pi(s')\big], \quad (27)$$

the time slot index t is omitted in the subsequent analysis to enhance notational clarity.

For any two Bellman operators $\mathcal{B}_\pi V_\varphi^\pi(s)$ and $\mathcal{B}_\pi \hat{V}_\varphi^\pi(s)$, we have

$$\begin{aligned}
&\left|\mathcal{B}_\pi V_\varphi^\pi(s) - \mathcal{B}_\pi \hat{V}_\varphi^\pi(s)\right| \\
&= |\mathbb{E}_{a\sim\pi(a|s)}\Big[r(s,a) \ + \gamma \mathbb{E}_{s'\sim p(s'|s,a)}[V_\varphi^\pi(s')] \\
&\quad - r(s,a) \ - \gamma \mathbb{E}_{s'\sim p(s'|s,a)}[\hat{V}_\varphi^\pi(s')]\Big]| \\
&= \gamma\left|\mathbb{E}_{a\sim\pi(a|s)}\Big[\mathbb{E}_{s'\sim p(s'|s,a)}\big[V_\varphi^\pi(s') - V_2(s')\big]\Big]\right| \\
&= \gamma\left|\mathbb{E}_{a\sim\pi(a|s),s'\sim p(s'|s,a)}\Big[V_\varphi^\pi(s') - \hat{V}_\varphi^\pi(s')\Big]\right| \\
&\le \gamma \mathbb{E}_{a\sim\pi(a|s),s'\sim p(s'|s,a)}\Big[\max_s\Big|V_\varphi^\pi(s) - \hat{V}_\varphi^\pi(s)\Big|\Big] \\
&= \gamma \mathbb{E}_{a\sim\pi(a|s),s'\sim p(s'|s,a)}\Big[\|V_\varphi^\pi - \hat{V}_\varphi^\pi\|_\infty\Big] \\
&= \gamma\|V_\varphi^\pi - \hat{V}_\varphi^\pi\|_\infty \quad \gamma \in [0,1] \qquad (28)
\end{aligned}$$

where $\|\cdot\|_\infty$ is L-Infinity Norm.

Therefore, for all states $s \in \mathbb{S}$, the inequality $|\mathcal{B}_\pi V_\varphi^\pi(s) - \mathcal{B}_\pi \hat{V}_\varphi^\pi(s)| \le \gamma\|V_\varphi^\pi - \hat{V}_\varphi^\pi\|_\infty$ holds, which implies that the Bellman operator $\mathcal{B}_\pi V_\varphi^\pi(s)$ is a contraction mapping. Therefore, by the Banach fixed-point theorem, starting from any initial value function $\tilde{V}_\varphi^\pi$, the Bellman operator $\mathcal{B}^\pi$ admits a unique fixed point, satisfying $\mathcal{B}^\pi V_\varphi^\pi = V_\varphi^\pi$. In other words, since the Bellman operator $\mathcal{B}_\pi V_\varphi^\pi$ is a contraction mapping, it follows

$$\begin{aligned}
&\|B_\pi^{n+1} V_\varphi^\pi - B_\pi^n V_\varphi^\pi\|_\infty \le \gamma\|B_\pi^n V_\varphi^\pi - B_\pi^{n-1} V_\varphi^\pi\|_\infty \\
&\le \gamma^2\|B_\pi^{n-1} V_\varphi^\pi - B_\pi^{n-2} V_\varphi^\pi\|_\infty \cdots \le \gamma^n\|B_\pi V_\varphi^\pi - V_\varphi^\pi\|_\infty
\end{aligned}$$

that repeatedly applying the Bellman operator $\mathcal{B}_\pi V_\varphi^\pi$ to any initial value function $\tilde{V}_\varphi^\pi$ ensures the difference between $B_\pi^{n+1} V_\varphi^\pi$ and $B_\pi^n V_\varphi^\pi$ converges to zero as n tends to infinity. Then, the sequence $[V, \mathcal{B}_\pi^1 V_\varphi^\pi, \mathcal{B}_\pi^2 V_\varphi^\pi, \ldots, \mathcal{B}_\pi^n V_\varphi^\pi]$ converges to the unique fixed point V_φ^π, i.e., $\lim_{n\to\infty} \mathcal{B}_\pi^n V_\varphi^\pi = V_\varphi^\pi$.

During the policy improvement phase, the policy is updated to a greedy policy π with respect to V_φ^π. In this paper, the policy is greedily updated by maximizing a value function V_φ^π-based generalized advantage (equation (19)). Since each application of the Bellman operator guarantees a monotonically non-decreasing update to the current value function, this approach ensures monotonic policy improvement, ultimately converging to an optimal policy.

However, in practice, performing an infinite number of Bellman operator iterations is infeasible. To address this, the proposed scheme incorporates several techniques to accelerate convergence, such as the trust region constraints for policies, the objective function clipper mechanism, the monotonic improvement of the objective function, the importance sampling and variance control, and the effectiveness of the advantage function, and has good convergence, as evidenced by the results of subsequent simulation experiments in this paper.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:08:48 UTC from IEEE Xplore. Restrictions apply.

TABLE I
LIST OF PARAMETERS

Parameter	Value
Size of experience replay buffer N_{max}	1024
Discount factor γ	0.9
Trade-off parameters λ for bias and variance	0.95
Number of updates of network per episode C	10
Hyperparameter for clipping in the policy objective σ	0.2
Learning rate of critic network	10^{-3}
Learning rate of policy network	1^{-4}
Number of training episodes N_{epi}	2500
Number of time slots per episode T	200
Transmit antenna gain g_T	5 dB
Receive antenna gain g_R	15 dB
Path loss index p	2.2
Signal wavelength λ_C	0.1 m
Chip parameters g_U of the UAV	10^{-19}

V. SIMULATION RESULTS

In this section, the effectiveness and superiority of the proposed SaHPPO-based scheme is verified through simulation experiments.

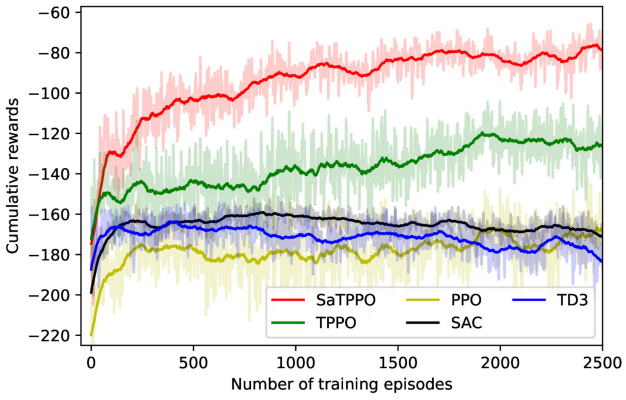
A. Simulation Setup and Baseline Schemes

1) Simulation Setup: To demonstarte UAV-enabled edge computing system based on network slicing techniques, we consider a smart campus scenario oriented to a square area with a side length of 1 km, where M = 20 users are distributed on the ground and a UAV in the sky provides computational offloading services to ground users. The flight altitude range of the UAV is 20-100 meters. The total computational power of the UAV is F=12 GHz, and the total available bandwidth is W=4 MHz. The users have different types of computational tasks that need to be offloaded to the UAV for processing. These heterogeneous computational tasks have different data sizes, computational complexity, and delay constraints, and the types of these computational tasks dynamically change with the user's behaviors at different time slots. The proposed SaHPPO-based scheme has a fully connected layer with two hidden layers for both the long-timescale and short-timescale policy network structure and a fully connected layer with three hidden layers for the critic network structure. Other parameters are detailed in Table I.

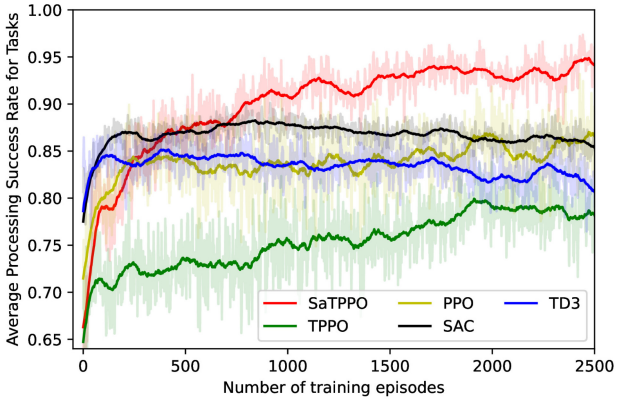
2) Baseline Schemes: In order to verify the superiority of the proposed scheme, we first choose the two-timescale PPO (TPPO) as the baseline scheme, which differs from the proposed SaTPPO-based scheme in that it has a fixed network slicing time window size. In addition, we choose three state-of-the-art reinforcement learning algorithms as the baseline schemes, namely PPO, soft actor critic (SAC), and twin delayed deep deterministic policy gradient (TD3), which are not based on two-timescale optimization.

B. Experiment Results

1) Comparison of Cumulated Reward and Average Processing Success Rate With Baselines: First, we compare the results of the proposed SaTPPO-based scheme versus the baseline schemes. We present the curves of the cumulative reward (negative value of the total system cost) and the average processing success rate of the computational tasks with the number of training episodes in Fig. 5 (a) and (b), respectively. The results show that the proposed strategy exhibits the highest cumulative reward value (i.e., achieves the lowest total system cost) at most stages of the training process compared to the baseline strategies. Meanwhile, although the average processing success rate of computational tasks of the proposed SaTPPO-based scheme is not optimal at the beginning of the training, it eventually converges to the highest average processing success rate of computational tasks among all schemes after sufficient training iterations.



(a)



(b)

Fig. 5. The curves of the cumulative reward and the average processing success rate of the computational tasks with the number of training episodes for the proposed scheme versus several baseline schemes. (a) Cumulative reward. (b) Average processing success rate.

The core reason why the baseline schemes PPO, TD3, and SAC are significantly lower than TPPO and the proposed SaTPPO scheme in terms of cumulative rewards is that they do not employ a two-timescale optimization strategy. These schemes reconfigure the resource allocation of network slices at each time step, which leads to a higher reconfiguration cost. In contrast, although the TPPO scheme introduces two-timescale optimization, which reduces the reconfiguration cost of network slices to a certain extent, the size of the time window of its network slices is fixed, and this limitation makes it difficult for the TPPO scheme to effectively meet the dynamic demands of user task processing while reducing the reconfiguration cost. Therefore, the TPPO scheme yields the lowest average processing success rate.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:08:48 UTC from IEEE Xplore. Restrictions apply.

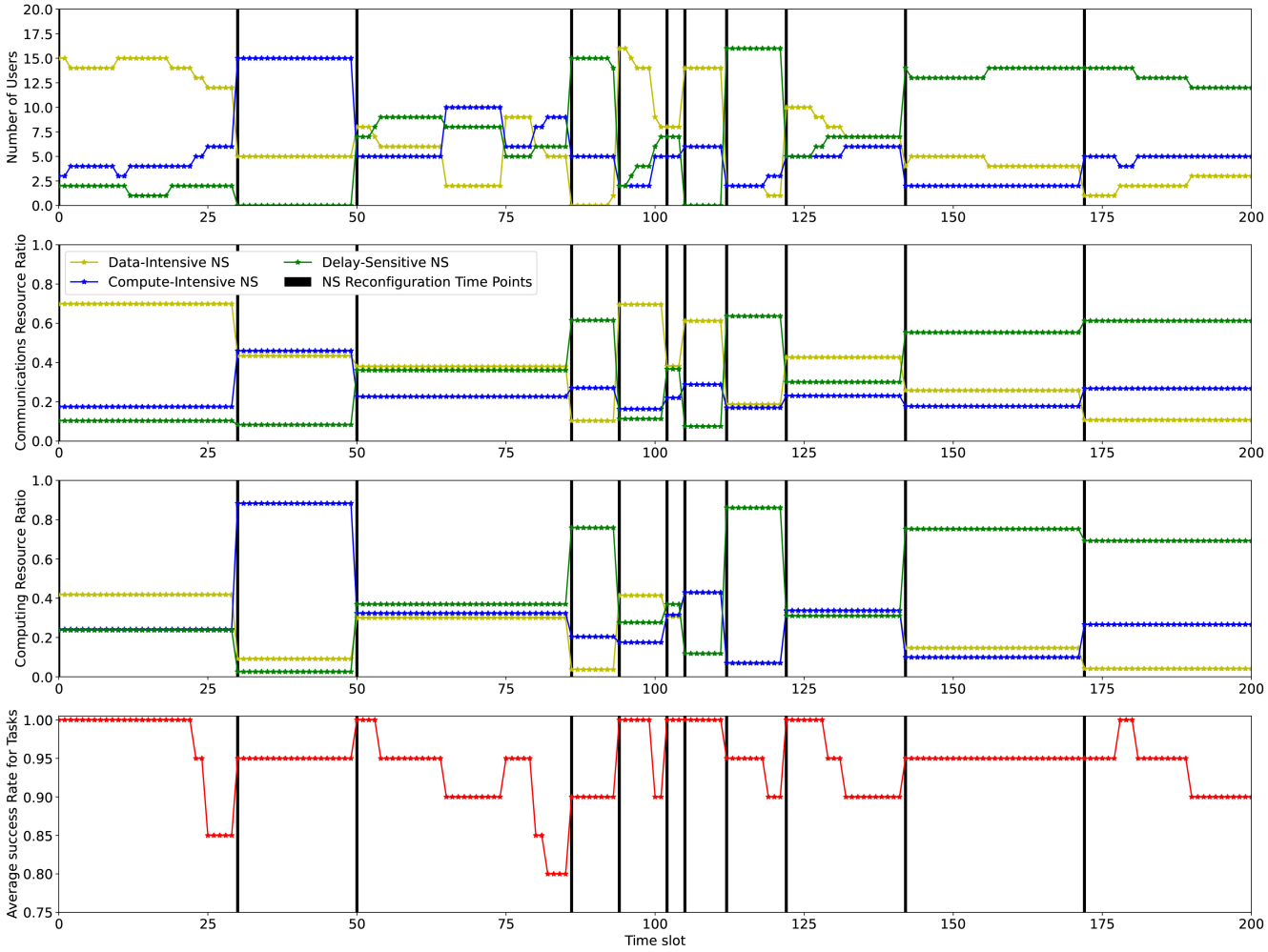


Fig. 6. The curves of the number of users, the ratio of allocated communication and computation resources for the three network slices, and the average processing success rate of computation tasks in terms of the number of time slots during an episode of the testing phase of the proposed scheme.

The proposed SaTPPO scheme outperforms TPPO in terms of cumulative rewards mainly due to its innovative self-adjustment mechanism for network slice resource allocation. The mechanism utilizes the evaluation network to predict the benefits of network slice reconfiguration and compares them with the reconfiguration costs, thus realizing the intelligent adjustment of network slice resource allocation. This means that the proposed SaTPPO-based scheme is able to flexibly adjust the time window size of network slices according to the dynamic evolution of the user's computing task types, and reconfigure the network slice resource allocation at the appropriate time nodes. This mechanism not only effectively reduces the system cost, but also significantly improves the processing efficiency of user computing tasks. In addition, for a fairer comparison, we apply the average network slice time window size of the SaTPPO scheme equal to that of the TPPO scheme after training to ensure that both are roughly equal in terms of network slice reconfiguration cost.

2) Performance Metrics of the Proposed Scheme: In Fig. 6, the curves of the number of users, the ratio of allocated communication and computation resources for the three network slices, and the average processing success rate of computation tasks (in terms of the number of time slots during an episode of the testing phase) of the proposed scheme are presented. It can be seen that the time window size of the network slices is flexible with the dynamic evolution of the number of users over time for the three network slices. Specifically, the time window size of the network slices is smaller when the dynamic evolution of the number of users over time for the network slices is faster, and larger otherwise. In addition, it can be seen that the average time window size of the network slices is about 20 time slots, and the processing success rate of the users' computation tasks is greater than 80% in all the time slots, which indicates that the proposed scheme can reconfigure the network slices' resource allocation at the appropriate time points (i.e., the time points where the black bars are located in the figure), and reallocate the communication resources and the computation resources for them, so that it can be used to effectively satisfy the users' heterogeneous and dynamic computing tasks while reducing the reconfiguration cost of network slices.

3) Impacts of Time Window Sizes of Network Slices: Further, we detail the specific impact of different network slice time window sizes on the system performance in the baseline scheme TPPO in Table II. The results show that with the increasing time window size, the number of network slice

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:08:48 UTC from IEEE Xplore. Restrictions apply.

TABLE II
DIFFERENT TIME WINDOW SIZES OF NS FOR TPPO ON SYSTEM PERFORMANCE

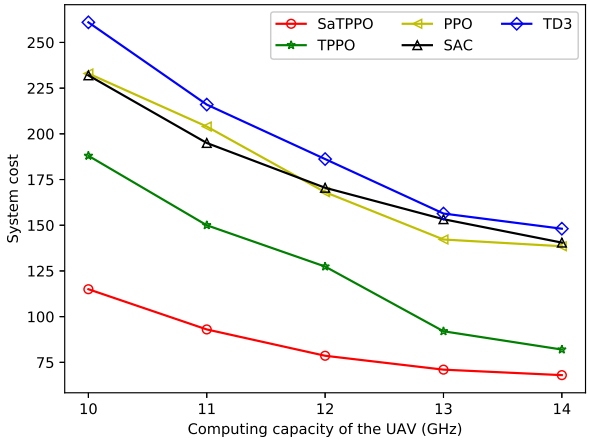
Time window sizes (time slots)	1	4	10	15	20	40	50	100	200	**Self-Adjusting**
Number of NS reconfigurations	200	50	20	13	10	5	4	2	1	10
Processing success rate of tasks	0.866	0.838	0.804	0.795	0.782	0.747	0.736	0.724	0.701	**0.939**
System Costs	187.1	115.1	120.5	124.2	127.4	140.3	131.6	133.8	132.7	**78.6**

reconfigurations shows a decreasing trend, the processing success rate of the computation tasks decreases, while the system cost shows a general trend of decreasing and then increasing. This phenomenon is due to the fact that the expansion of the time window reduces the number of reconfigurations and the corresponding reconfiguration cost, but also makes it difficult for the resource allocation of network slices to fully adapt to the dynamically changing resource requirements of computation tasks, which leads to the decline in the success rate of computation tasks and the increase in the cost due to the failure of computation tasks. This observation reveals the tradeoff between resource optimization granularity and deployment costs.

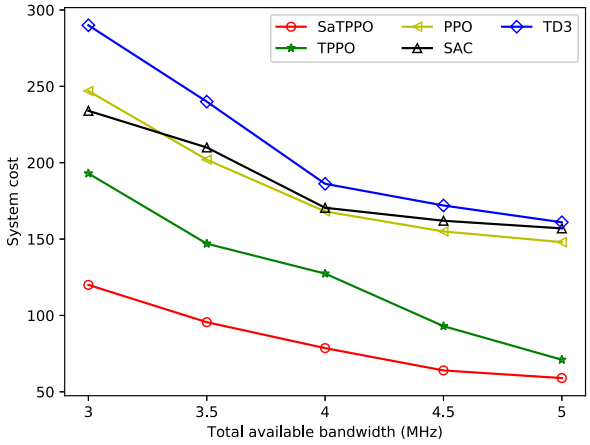
To address this issue, our proposed SaTPPO-based scheme can adaptively adjust the time window of network slicing according to the dynamically changing resource demands of computing tasks. The scheme not only maximizes the success rate of computing task processing, but also ensures the optimization of system cost. In addition, it is worth noting that compared to the TPPO scheme with a fixed time window of 1-time slot (i.e., the PPO scheme), the computational task processing success rate of our proposed scheme is higher. This is mainly because, according to Eq. (7), the network slice reconfiguration cost is closely related to the increase in slice resources, so the PPO scheme tends to reduce the increase in slice resources at each reconfiguration in order to reduce the reconfiguration cost, at the expense of slightly reducing the processing success rate of the computation tasks. Therefore, the proposed scheme manages to draw a balance between deployment costs and processing success rate.

4) Impacts of Computational and Communication Resources on System Cost: Fig. 7 (a) and (b) show how the computing capacity of UAV and total available bandwidth affect the system cost, respectively. It can be seen that as the computational power of UAV and the total available bandwidth increase, the system cost of both the proposed SaTPPO-based scheme and baseline schemes show a decreasing trend, but the proposed scheme has a clear advantage in different scenarios with different computational power of UAV and total available bandwidth, and the system cost is the lowest in all of them.

It is further observed that as the computational and bandwidth resources increase, the advantages of the proposed algorithms relative to TPPO presumably show a progressively smaller trend, mainly because the increase in resources makes the fixed network slicing time window approach of TPPO also better in meeting the resource requirements of the computational tasks. Nevertheless, when comparing the three non-two-timescale schemes (i.e., PPO, SAC, and TD3), the advantage of the SaTPPO algorithm remains significant. This is because although the increase in resources can improve



(a)



(b)

Fig. 7. Computing capacity of the UAV and total available bandwidth versus system cost. (a) Computing capacity of the UAV. (b) Total available bandwidth.

the execution success rate of the computational tasks of these three baseline schemes, it is not able to significantly reduce the system cost by reducing the number of network slice reconfigurations as the proposed scheme.

VI. CONCLUSION

In this paper, we focus on the dynamic heterogeneous computing task offloading problem for UAV-enabled edge computing based on network slicing techniques, aiming to minimize the total system cost. Given that over-frequent network slicing reconfiguration increases the system cost substantially while over-slow network slicing reconfiguration fails to adapt to the time-varying demands, we propose

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:08:48 UTC from IEEE Xplore. Restrictions apply.

an efficient self-adjusting two-timescale PPO-based (i.e., SaTPPO) scheme to optimize network slicing management. Specifically, the proposed scheme optimizes the user-level resource allocation and UAV trajectory planning problems on short timescales, and optimizes the slice-level resource allocation problem on long timescales, thereby significantly reducing the reconfiguration cost of network slices while efficiently handling computational tasks. In addition, the proposed scheme utilizes the property of the critic network that fits the state-value function to predict the reconfiguration gain of the network slices, thus guiding the network slices to be reconfigured at the appropriate time, and realizing the self-adjustment of the size of the slicing time window. Finally, this paper conducts extensive experiments in the smart campus scenario to verify the effectiveness and superiority of the proposed scheme from different perspectives. It is worth noting that the proposed scheme is not only applicable to the specific scenarios explored in this paper, but also provides an effective method for solving the adaptive adjustment of time windows in a wider range of two-timescale optimization problems.

REFERENCES

[1] C.-X. Wang et al., "On the road to 6G: Visions, requirements, key technologies, and testbeds," *IEEE Commun. Surveys Tuts.*, vol. 25, no. 2, pp. 905–974, 2nd Quart., 2023.

[2] A. R. Hossain, W. Liu, N. Ansari, A. Kiani, and T. Saboorian, "AI-native end-to-end network slicing for next-generation mission-critical services," *IEEE Trans. Cogn. Commun. Netw.*, vol. 11, no. 1, pp. 48–58, Feb. 2025.

[3] Y. Shi et al., "Machine learning for large-scale optimization in 6G wireless networks," *IEEE Commun. Surveys Tuts.*, vol. 25, no. 4, pp. 2088–2132, 4th Quart., 2023.

[4] Y. Zhao et al., "Joint content caching, service placement, and task offloading in UAV-enabled mobile edge computing networks," *IEEE J. Sel. Areas Commun.*, vol. 43, no. 1, pp. 51–63, Jan. 2025.

[5] H. Djigal, J. Xu, L. Liu, and Y. Zhang, "Machine and deep learning for resource allocation in multi-access edge computing: A survey," *IEEE Commun. Surveys Tuts.*, vol. 24, no. 4, pp. 2449–2494, 4th Quart., 2022.

[6] H. A. Alameddine, S. Sharafeddine, S. Sebbah, S. Ayoubi, and C. Assi, "Dynamic task offloading and scheduling for low-latency IoT services in multi-access edge computing," *IEEE J. Sel. Areas Commun.*, vol. 37, no. 3, pp. 668–682, Mar. 2019.

[7] L. Wang, K. Wang, C. Pan, W. Xu, N. Aslam, and L. Hanzo, "Multi-agent deep reinforcement learning-based trajectory planning for multi-UAV assisted mobile edge computing," *IEEE Trans. Cogn. Commun. Netw.*, vol. 7, no. 1, pp. 73–84, Mar. 2021.

[8] Z. Ullah, F. Al-Turjman, and L. Mostarda, "Cognition in UAV-aided 5G and beyond communications: A survey," *IEEE Trans. Cogn. Commun. Netw.*, vol. 6, no. 3, pp. 872–891, Sep. 2020.

[9] Y. Wu, H.-N. Dai, H. Wang, Z. Xiong, and S. Guo, "A survey of intelligent network slicing management for industrial IoT: Integrated approaches for smart transportation, smart energy, and smart factory," *IEEE Commun. Surveys Tuts.*, vol. 24, no. 2, pp. 1175–1211, 2nd Quart., 2022.

[10] S. Jiang, J. Zheng, F. Yan, and S. Zhao, "Reinforcement-learning-based network slicing and resource allocation for multi-access edge computing networks," *IEEE Trans. Cogn. Commun. Netw.*, vol. 10, no. 3, pp. 1132–1145, Jun. 2024.

[11] J. Khamse-Ashari, G. Senarath, I. Bor-Yaliniz, and H. Yanikomeroglu, "An agile and distributed mechanism for inter-domain network slicing in next generation mobile networks," *IEEE Trans. Mobile Comput.*, vol. 21, no. 10, pp. 3486–3501, Oct. 2022.

[12] J. Hwang, L. Nkenyereye, N. Sung, J. Kim, and J. Song, "IoT service slicing and task offloading for edge computing," 2020, *arXiv:2008.10210*.

[13] S. D'Oro, L. Bonati, F. Restuccia, M. Polese, M. Zorzi, and T. Melodia, "Sl-edge: Network slicing at the edge," in *Proc. 21st Int. Symp. Theory, Algorithmic Found., Protocol Design Mobile Netw. Mobile Comput.*, 2020, pp. 1–10.

[14] C. Tian, H. Cao, J. Xie, S. Garg, M. Alrashoud, and P. Tiwari, "Community detection-empowered self-adaptive network slicing in multi-tier edge-cloud system," *IEEE Trans. Netw. Service Manag.*, vol. 21, no. 3, pp. 2624–2636, Jun. 2024.

[15] G. Alkhoury, S. Berri, and A. Chorti, "Task offloading with 5G network slicing for V2X communications," in *Proc. IEEE Glob. Commun. Conf. (GLOBECOM)*, 2022, pp. 4425–4430.

[16] Q. Ye, W. Shi, K. Qu, H. He, W. Zhuang, and X. Shen, "Joint RAN slicing and computation offloading for autonomous vehicular networks: A learning-assisted hierarchical approach," *IEEE Open J. Veh. Technol.*, vol. 2, pp. 272–288, Jun. 2021. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/9454395

[17] H. Shen, Y. Tian, T. Wang, and G. Bai, "Slicing-based task offloading in space-air-ground integrated vehicular networks," *IEEE Trans. Mobile Comput.*, vol. 23, no. 5, pp. 4009–4024, May 2024.

[18] T. Huang, Z. Fang, Q. Tang, R. Xie, T. Chen, and F. R. Yu, "Dual-timescales optimization of task scheduling and resource slicing in satellite-terrestrial edge computing networks," *IEEE Trans. Mobile Comput.*, vol. 23, no. 12, pp. 14111–14126, Dec. 2024.

[19] G. Alkhoury, S. Berri, and A. Chorti, "Deep reinforcement learning-based network slicing algorithm for 5G heterogenous services," in *Proc. IEEE Glob. Commun. Conf. (GLOBECOM)*, 2023, pp. 5190–5195.

[20] Y. Chiang, C.-H. Hsu, G.-H. Chen, and H.-Y. Wei, "Deep Q-learning-based dynamic network slicing and task offloading in edge network," *IEEE Trans. Netw. Service Manag.*, vol. 20, no. 1, pp. 369–384, Mar. 2023.

[21] S. Jošilo and G. Dán, "Joint wireless and edge computing resource management with dynamic network slice selection," *IEEE/ACM Trans. Netw.*, vol. 30, no. 4, pp. 1865–1878, Aug. 2022.

[22] Y. Zhang, H. Zhang, Y. Li, and S. Yuan, "Joint resource allocation and task offloading for hybrid NOMA-assisted MEC network with network slicing," in *Proc. 25th Int. Conf. Adv. Commun. Technol. (ICACT)*, 2023, pp. 164–170.

[23] Z. Wang, Y. Wei, Z. Feng, F. R. Yu, and Z. Han, "Resource management and reflection optimization for intelligent reflecting surface assisted multi-access edge computing using deep reinforcement learning," *IEEE Trans. Wireless Commun.*, vol. 22, no. 2, pp. 1175–1186, Feb. 2023.

[24] M.-M. Zhao, K. Xu, Y. Cai, Y. Niu, and L. Hanzo, "Secrecy rate maximization of RIS-assisted SWIPT systems: A two-timescale beamforming design approach," *IEEE Trans. Wireless Commun.*, vol. 22, no. 7, pp. 4489–4504, Jul. 2022.

[25] L. Liu, Z. Zhang, H. Zhang, Y. Zhang, and Y. Xu, "Two-timescale dynamic resource management in smart-grid powered heterogeneous cellular networks," *IEEE Trans. Wireless Commun.*, vol. 23, no. 4, pp. 2681–2695, Apr. 2024.

[26] D. Han et al., "Two-timescale learning-based task offloading for remote IoT in integrated satellite–terrestrial networks," *IEEE Internet Things J.*, vol. 10, no. 12, pp. 10131–10145, Jun. 2023.

[27] G. Ding, J. Yuan, G. Yu, and Y. Jiang, "Two-timescale resource management for ultrareliable and low-latency vehicular communications," *IEEE Trans. Commun.*, vol. 70, no. 5, pp. 3282–3294, May 2022.

[28] Y. Shi, C. Yi, R. Wang, Q. Wu, B. Chen, and J. Cai, "Service migration or task rerouting: A two-timescale online resource optimization for MEC," *IEEE Trans. Wireless Commun.*, vol. 23, no. 2, pp. 1503–1519, Feb. 2024.

[29] T. Zhang, Y. Wang, W. Yi, Y. Liu, C. Feng, and A. Nallanathan, "Two time-scale caching placement and user association in dynamic cellular networks," *IEEE Trans. Commun.*, vol. 70, no. 4, pp. 2561–2574, Apr. 2022.

[30] Y. Yang et al., "Dynamic human digital twin deployment at the edge for task execution: A two-timescale accuracy-aware online optimization," *IEEE Trans. Mobile Comput.*, vol. 23, no. 12, pp. 12262–12279, Dec. 2024.

[31] Y. He et al., "Two-timescale resource allocation for automated networks in IIoT," *IEEE Trans. Wireless Commun.*, vol. 21, no. 10, pp. 7881–7896, Oct. 2022.

[32] Y. Shi, Y. Yang, C. Yi, B. Chen, and J. Cai, "Toward online reliability-enhanced microservice deployment with layer sharing in edge computing," *IEEE Internet Things J.*, vol. 11, no. 13, pp. 23370–23383, Jul. 2024.

[33] J. Li et al., "A reinforcement learning-based stochastic game for energy-efficient UAV swarm-assisted MEC with dynamic clustering and scheduling," *IEEE Trans. Green Commun. Netw.*, vol. 9, no. 1, pp. 255–270, Mar. 2025.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:08:48 UTC from IEEE Xplore. Restrictions apply.

[34] X. Li, X. Zhang, and T. Huang, "Joint task offloading and service placement for mobile edge computing: An online two-timescale approach," *IEEE Trans. Cloud Comput.*, vol. 11, no. 4, pp. 3656–3671, Oct.–Dec. 2023.
[35] X. Li, Y. Qin, J. Huo, and W. Huangfu, "Computation offloading and trajectory planning of multi-UAV-enabled MEC: A knowledge-assisted multiagent reinforcement learning approach," *IEEE Trans. Veh. Technol.*, vol. 73, no. 5, pp. 7077–7088, May 2024.
[36] Y. Cui, H. Shi, R. Wang, P. He, D. Wu, and X. Huang, "Multi-agent reinforcement learning for slicing resource allocation in vehicular networks," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 2, pp. 2005–2016, Feb. 2024.



Xulong Li (Student Member, IEEE) is currently pursuing the Ph.D. degree in information and communication engineering with the School of Computer and Communication Engineering, University of Science and Technology Beijing. His current research interests include mobile edge computing, the Internet of Things, and deep reinforcement learning.



Wencan Mao (Member, IEEE) received the B.E. degree in vehicle engineering from the Wuhan University of Technology, Wuhan, China, in 2017 and the M.S. degree in mechanical engineering and the Ph.D. degree from the Department of Information and Communications Engineering, Aalto University, Espoo, Finland, in 2019. She is currently working as a Postdoctoral Researcher with the National Institute of Informatics, Tokyo, Japan. Her current research interests include reinforcement learning, large language models, unmanned aerial vehicles, vehicle-to-everything, edge computing, smart and sustainable cities, integrated sensing and communications, and the Internet of Things.



Xinyi Xu (Member, IEEE) received the B.S. degree in electronic science and technology and the Ph.D. degree from the School of Electronic Engineering, Xidian University, China, in 2014 and 2021, respectively. She is currently working as an Associate Professor with the School of Computer and Communication Engineering, University of Science and Technology Beijing, China. Her research interests include computer vision, machine learning, and the applications of deep learning.



Yaxi Liu (Member, IEEE) received the B.S. and Ph.D. degrees in communication engineering from the School of Computer and Communication Engineering, University of Science and Technology Beijing, Beijing, China, in 2016 and 2021, respectively, where she is currently a Lecturer and a Postdoctor from 2021 to 2023. Her research interests include wireless communications, integrated sensing and communications, network optimization, and planning. She received the 2018 IEEE WCSP Best Paper Award.



Huifeng Zhang received the B.S. degree in computer science and technology from Shanxi Datong University, China, in 2014, and the Ph.D. degree in computer science and technology from the Dalian University of Technology, China, in 2021. From 2019 to 2020, she was a Visiting Scholar with Florida International University, USA. She is currently an Assistant Researcher with Zhejiang Lab, China. Her current research interests include topological structure and analysis of interconnection networks, graph theory, and network optimization.



Wei Huangfu (Member, IEEE) received the M.S. and Ph.D. degrees in electronic engineering from Tsinghua University, Beijing, China, in 1998 and 2001, respectively. He is currently a Full Professor with the School of Computer and Communication Engineering, University of Science and Technology Beijing. His main research interests include statistical signal processing, the Internet of Things, cooperative communications networks, and wireless sensor network.



Keping Long (Senior Member, IEEE) received the M.S. and Ph.D. degrees in electric circuit and system from University of Electronic Science and Technology of China, Chengdu, China, in 1995 and 1998, respectively. He is currently a Professor with the School of Computer and Communication Engineering, University of Science and Technology Beijing. His main research interests include statistical signal processing, channel estimation in multiple-input multiple-output orthogonal frequency division multiplexing systems, cooperative communications, and computer networks.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:08:48 UTC from IEEE Xplore. Restrictions apply.